RELATÓRIO DE CONTINUIDADE

PreJud - Dashboard do Freelancer

Resumo técnico do que foi feito nesta sessão, quais arquivos foram alterados, quais problemas foram corrigidos e exatamente de onde devemos continuar a partir do comando 102.

PROJETO	MÓDULO	STATUS ATUAL	PRÓXIMO MARCO
PreJud SaaS	**Dashboard / Functions**	**Estabilizado parcialmente**	**Comando 102 em diante**

1. Objetivo da sessão

O objetivo foi sair de uma dashboard que lia dados direto do Firestore, sem Cloud Functions implementadas, e levar o projeto para um ponto funcional com backend agregador, integração inicial da dashboard e correção dos principais erros que impediam uso e validação.

> Nesta sessão o foco real deixou de ser apenas layout e passou a ser: estruturar Functions, integrar a dashboard ao backend, estabilizar a leitura dos acordos e corrigir o fluxo de aceite para que o acordo confirmado apareça corretamente na visão do emissor.

2. O que foi feito

Backend / Functions

- Inicializada a estrutura de **Firebase Functions** em TypeScript.
- Criada a function `getFreelancerDashboard`.
- Implementada agregação de acordos, cobranças, histórico e prioridade.
- Adicionado retorno compatível com a dashboard antiga: `totalAgreements`, `activeAgreements`, `pendingConfirmation`, `pendingValue`, `paidValue`.

Frontend / Dashboard

- Integrado o frontend com **Cloud Functions**.
- Configurado `getFunctions` e `connectFunctionsEmulator`.
- Atualizado `dashboardService.ts` para usar a versão V2 via function.
- Atualizado `useFreelancerDashboard.ts` para consumir a V2.
- Corrigida conversão de datas recebidas da function para objetos `Date`.

- Corrigido retorno para não limitar acordos a apenas 5 durante o teste da dash.
- Adicionada ordenação por `createdAt desc` para trazer acordos mais recentes.

- Corrigida a fórmula de **Rascunhos** para não ficar negativa.
- Inserido o bloco de **Ações rápidas** dentro do `<main>` da dashboard.

3. Problemas encontrados e como foram resolvidos

3.1 - Não existia backend de Functions

No início, a pasta `functions` estava vazia. Foi necessário rodar a inicialização do Firebase Functions e criar do zero o arquivo principal `functions/src/index.ts`.

3.2 - Erro de CORS / chamada indo para produção

O frontend estava chamando a function de produção e falhando com CORS. Isso foi resolvido ao configurar o emulator de Functions no arquivo `frontend/lib/firebase.ts` e ao ativar o uso de emuladores no `.env.local`.

3.3 - Erros de data

Como a function devolvia campos de data em formato serializado, a UI que antes dependia do Firestore direto começou a quebrar com erros como: `toLocaleDateString is not a function` e `toISOString is not a function`.

Isso foi corrigido em dois pontos:

- Na V2 do `dashboardService.ts`, convertendo datas vindas da function para `Date`.
- Em `firebaseAgreementService.ts`, tornando seguro o cálculo do hash com `new Date(...).toISOString()`.

3.4 - JSX quebrado na dashboard

Durante a inserção manual do bloco de ações rápidas, alguns trechos ficaram fora do `return` e até dentro da região entre loading e error state. Isso gerou erros de parser.

O arquivo foi limpo e estabilizado via PowerShell, removendo blocos soltos e restaurando o **loading state**.

3.5 - Acordo aceito não aparecia na dashboard

Primeiro foi identificado um erro real no fluxo de confirmação: a função `confirm` estava salvando `status: 'active'`, mas o domínio correto do PreJud exige `status: 'confirmed'`.

Depois da correção, foi confirmado que o acordo já estava salvo como `confirmed`. O segundo fator que atrapalhava a visualização era a function da dashboard devolver só uma fatia limitada da lista e sem ordenação, o que foi corrigido com `orderBy("createdAt", "desc")` e remoção temporária do `slice(0, 5)` dos acordos.

> Também foi observado, pelas telas, que a dashboard foi aberta em contas diferentes em momentos distintos. Isso importa porque a dash filtra por `freelancerId == user.uid`.

4. Arquivos alterados nesta sessão

ARQUIVO	O QUE FOI ALTERADO	OBSERV
`C:\prejud-saas-new\functions\src\index.ts`	Criação da function `getFreelancerDashboard`, ordenação dos acordos, retorno expandido de stats, lista de acordos sem corte de 5 itens para teste.	É o cora dashbo
`C:\prejud-saas-new\frontend\lib\firebase.ts`	Adicionado suporte a `firebase/functions` e conexão com o emulator de Functions.	Hoje es configu usar ap emulat Functio Auth/Fi Emulat
`C:\prejud-saas-new\frontend\services\dashboardService.ts`	Criada a função `getFreelancerDashboardDataV2` para consumir a callable function e converter datas.	Mantér compat o forma esperac hook/d
`C:\prejud-saas-new\frontend\hooks\useFreelancerDashboard.ts`	Hook alterada para chamar a V2 da dashboard.	Continu estrutu loading
`C:\prejud-saas-new\frontend\app\(dashboard)\dashboard\page.tsx`	Correções de JSX, integração com dados novos e correção da fórmula de rascunhos.	O bloco rápidas dentro

`C:\prejud-saas-new\frontend\services\agreementService.ts`	Correção do status no aceite do cliente: `active` -> `confirmed`.	Essenci acordo ativo na
`C:\prejud-saas-new\frontend\services\firebaseAgreementService.ts`	Correção no cálculo do hash para converter `createdAt` com segurança.	Evita er do acor
`C:\prejud-saas-new\frontend\.env.local`	Ativado uso de emuladores durante desenvolvimento.	Necess fluxo lo functio

5. Estado atual do projeto

O que já está funcionando

- Dashboard do freelancer abrindo com backend via function.
- Leitura de acordos do emissor.
- Correção do aceite para `confirmed`.
- Visualização do acordo em página interna sem erro de data.
- Emulator de Functions operacional.

O que ainda falta

- Fazer os botões da dashboard abrirem a **página formal** do acordo, não só a página interna.
- Reaproveitar a tela pública `/p/[id]` como fonte de verdade visual do acordo com timeline e hash.
- Estruturar histórico recente com base real em timeline / notices.
- Profissionalizar dark mode / light mode sem duplicar componentes.

6. Onde paramos exatamente

O último ponto funcional da sessão foi:

> **Comando 102** - localizar a página pública `/p/[id]`, porque o comportamento esperado para **Ver detalhes** e **Abrir acordo** é abrir o formato formal do acordo com timeline e hash, no padrão do link público: `/p/BRWt2iLDLLfKPWpRIzbe?t=ea30a26b-a2e4-4d49-a3db-45e9664d2fd1`.

Ou seja: a dashboard já voltou a mostrar o acordo, mas os links ainda não estão apontando para a visualização formal desejada.

7. O que precisa ser feito a partir do comando 102

1. **Localizar a página pública do acordo**
   Encontrar o arquivo real da rota `/p/[id]` dentro de `frontend/app`. A meta é reaproveitar exatamente a tela que já exibe acordo com hash e timeline.

2. **Analisar quais dados essa página exige**
   Confirmar se ela depende de `clientAccessToken`, timeline completa, hash, eventos, notices ou campos adicionais que ainda não estejam indo para a dashboard.

3. **Decidir o comportamento correto dos botões da dashboard**
   Ajustar **Ver detalhes** e **Abrir acordo** para:

   - 4. ou redirecionarem para a página pública `/p/[id]?t=token` quando houver token;
   - 5. ou criarem uma página interna espelho com a mesma estrutura visual formal.

6. **Levar o token para a dashboard**
   Hoje a dashboard mapeia acordos para a UI sem necessariamente usar o `clientAccessToken`. Será preciso garantir que esse campo esteja disponível no mapeamento usado pelos botões.

7. **Revisar timeline e hash**
   Se a página pública é a referência visual correta, então a timeline e o hash devem ser tratados como parte central da experiência. A dashboard deve abrir para esse fluxo, não para uma página simplificada.

8. **Depois disso, retomar refinamentos da dashboard**
   Somente após os links estarem certos, continuar com:

   - 9. histórico recente real;
   - 10. ações rápidas conectadas a funções reais;
   - 11. modo claro / escuro profissional.

8. Comando de retomada sugerido

Para continuar exatamente do ponto onde paramos, o primeiro comando sugerido é:

```
Get-ChildItem -Path "C:\prejud-saas-new\frontend\app" -Recurse -File | Where-Object { $_.FullName -match "\\p\\[id\]\page\.tsx$" } | Select-Object FullName
```

Esse comando localiza a rota pública do acordo e nos coloca de volta exatamente no ponto do comando 102.

Relatório gerado para continuidade técnica do projeto PreJud. Este HTML foi preparado para impressão em PDF sem perda de estrutura.